

Universidad Autónoma de Zacatecas
Unidad Académica de Ingeniería Eléctrica
Ingeniería de Software

TRABAJO FINAL
Introducción a la Programación

■ **TIENDA DE VIDEOJUEGOS** ■

Docente:	Santiago Esparza Guerrero
Alumno:	Emilio Juan Antonio Juarez Torres
Fecha de entrega:	19 de mayo de 2026
Líder del proyecto:	Emilio Juan Antonio Juarez Torres

2. Descripción del Programa

¿Qué hace el programa?

La Tienda de Videojuegos es una aplicación de escritorio desarrollada en Python con interfaz gráfica (GUI) mediante la biblioteca Tkinter. Permite gestionar de forma integral el inventario de una tienda de videojuegos y llevar un registro detallado de cada venta realizada.

¿Cuál es su finalidad?

Ofrecer al administrador de una tienda una herramienta sencilla, visual e intuitiva que centralice las operaciones de inventario y ventas, eliminando el uso de registros manuales en papel y reduciendo errores humanos.

¿Qué problema busca resolver?

En pequeños negocios de videojuegos, el control de existencias y ventas suele hacerse de manera informal. Este programa resuelve la necesidad de llevar un inventario actualizado en tiempo real, registrar ventas con fecha y hora, y evitar la venta de productos sin stock disponible.

Requerimientos Funcionales

- Agregar productos con nombre, precio y cantidad al inventario.
- Visualizar el inventario completo en una tabla.
- Buscar un producto por nombre y ver su información.
- Registrar ventas: descuenta una unidad del inventario y guarda la transacción.
- Eliminar productos del catálogo.
- Mostrar el historial de ventas con nombre, precio y fecha/hora.

Requerimientos No Funcionales

- Interfaz gráfica amigable e intuitiva (Tkinter).
- Validación de campos obligatorios y tipos de datos.
- Mensajes de confirmación y error claros para el usuario.
- Código documentado y organizado.
- Compatible con Python 3.x en sistemas Windows, Linux y macOS.

3. Problema a Resolver

Una tienda pequeña de videojuegos lleva su inventario de manera manual en libretas o en archivos de texto sin estructura. Esto provoca pérdidas de información, errores al calcular el stock disponible y dificultad para recuperar el historial de ventas. El programa propone automatizar este proceso.

Definición del problema

Entradas	Proceso	Salidas
Nombre del videojuego	Validar y almacenar datos en lista de inventario	Inventario actualizado en tabla
Precio del videojuego	Descontar stock al registrar venta	Historial de ventas con fecha/hora
Cantidad en stock	Buscar producto por nombre	Mensajes de confirmación o error
Accion del usuario (agregar, buscar, vender, eliminar)	Eliminar producto del catalogo	Catalogo sin el producto eliminado

El problema cumple con los lineamientos del curso: es realista, cotidiano, de complejidad moderada, y tiene entradas, proceso y salidas claramente definidos.

4. Herramientas Utilizadas

Herramienta	Tipo	Uso en el proyecto
Python 3.x	Lenguaje de programacion	Desarrollo de toda la logica del programa
Tkinter	Biblioteca estandar Python	Construccion de la interfaz grafica (GUI)
ttk (Treeview)	Modulo de Tkinter	Tablas de inventario y ventas
datetime	Modulo estandar Python	Registro de fecha y hora en cada venta
messagebox	Modulo de Tkinter	Ventanas emergentes de confirmacion y error
VS Code	Entorno de desarrollo (IDE)	Escritura, edicion y prueba del codigo fuente
Inteligencia Artificial (apoyo)	Herramienta de apoyo (opcional)	Consultas de sintaxis y resolucion de dudas

Todas las herramientas utilizadas son de software libre y de código abierto, cumpliendo con los lineamientos de la asignatura.

A continuación se presenta el código fuente completo del programa, debidamente documentado con comentarios explicativos en cada sección relevante.

[illegible]

```

f"${venta['precio']}",
venta['fecha']
))

def buscar_producto():
    """Busca un producto por nombre (sin distincin de mayusculas) y muestra sus datos."""
    nombre = entry_buscar.get()
    for producto in inventario:
        if producto['nombre'].lower() == nombre.lower():
            messagebox.showinfo('Producto encontrado',
                f"Nombre: {producto['nombre']}\n"
                f"Precio: ${producto['precio']}\n"
                f"Cantidad: {producto['cantidad']}")
            return
    messagebox.showwarning('Error', 'Producto no encontrado')

def registrar_venta():
    """Registra una venta: descuenta 1 unidad y guarda la transaccion con timestamp."""
    nombre = entry_buscar.get()
    for producto in inventario:
        if producto['nombre'].lower() == nombre.lower():
            if producto['cantidad'] > 0:
                producto['cantidad'] -= 1
                venta = {
                    'nombre': producto['nombre'],
                    'precio': producto['precio'],
                    'fecha' : datetime.now().strftime('%d/%m/%Y %H:%M:%S')
                }
                ventas.append(venta)
            actualizar_tabla_inventario()
            actualizar_tabla_ventas()
            messagebox.showinfo('Venta', 'Venta registrada correctamente')
        else:
            messagebox.showwarning('Error', 'Producto agotado')
    return
    messagebox.showwarning('Error', 'Producto no encontrado')

def eliminar_producto():
    """Elimina un producto del inventario buscandolo por nombre."""
    nombre = entry_buscar.get()
    for producto in inventario:
        if producto['nombre'].lower() == nombre.lower():
            inventario.remove(producto)
            actualizar_tabla_inventario()
            messagebox.showinfo('Exito', 'Producto eliminado')
            return
    messagebox.showwarning('Error', 'Producto no encontrado')

def limpiar_campos():
    """Borra el contenido de los campos del formulario de alta de producto."""
    entry_nombre.delete(0, tk.END)
    entry_precio.delete(0, tk.END)

```

[illegible]

```
tabla_inventario.heading('Cantidad', text='Cantidad')
tabla_inventario.pack(fill='x', padx=20)

# Tabla de ventas
tk.Label(ventana, text='REGISTRO DE VENTAS', font=('Arial', 14, 'bold'),
bg='white').pack(pady=10)
tabla_ventas = ttk.Treeview(ventana,
columns=('Nombre', 'Precio', 'Fecha'),
show='headings', height=8)
tabla_ventas.heading('Nombre', text='Videojuego')
tabla_ventas.heading('Precio', text='Precio')
tabla_ventas.heading('Fecha', text='Fecha y Hora')
tabla_ventas.pack(fill='x', padx=20)
ventana.mainloop()
```


6. Pruebas del Sistema

A continuacion se presentan los casos de prueba definidos para validar el correcto funcionamiento del sistema. Cada caso especifica la entrada proporcionada, el resultado esperado y el resultado obtenido.

Caso	Entrada	Resultado esperado	Resultado obtenido
CP-01 Agregar producto valido	Nombre: The Last of Us Precio: 899.99 Cantidad: 5	Producto aparece en tabla de inventario y muestra mensaje de exito	CORRECTO - Producto agregado y tabla actualizada
CP-02 Campos vacios	Nombre: (vacio) Precio: (vacio) Cantidad: (vacía)	Advertencia: 'Completa todos los campos'	CORRECTO - Advertencia mostrada, no se agrega nada
CP-03 Precio no numerico	Nombre: GTA VI Precio: abc Cantidad: 3	Error: 'Precio o cantidad invalidos'	CORRECTO - Error capturado por bloque try/except
CP-04 Buscar producto existente	Busqueda: The Last of Us	Ventana emergente con nombre, precio y cantidad del producto	CORRECTO - Informacion correcta mostrada
CP-05 Buscar producto inexistente	Busqueda: Minecraft	Advertencia: 'Producto no encontrado'	CORRECTO - Advertencia mostrada al usuario
CP-06 Registrar venta con stock	Busqueda: The Last of Us (cantidad actual: 5)	Cantidad baja a 4, venta registrada en historial con fecha/hora	CORRECTO - Inventario y tabla de ventas actualizados
CP-07 Registrar venta sin stock	Busqueda: producto con cantidad=0	Advertencia: 'Producto agotado'	CORRECTO - No se registra venta ni se modifica inventario
CP-08 Eliminar producto	Busqueda: The Last of Us	Producto eliminado del inventario y tabla actualizada	CORRECTO - Producto removido correctamente
CP-09 Busqueda case-insensitive	Busqueda: 'the last of us' (minusculas)	Producto encontrado sin importar el uso de mayusculas	CORRECTO - Comparacion .lower() funciona correctamente

7. Evidencia Visual (Pantallas)

A continuacion se describen las pantallas principales del programa. Las capturas se obtuvieron ejecutando el programa en un sistema con fondo claro y resolucion adecuada para garantizar legibilidad.

Pantalla 1 – Ventana Principal

Al iniciar el programa se muestra la ventana principal con el titulo 'TIENDA DE VIDEOJUEGOS'. En la parte superior se encuentra el formulario de alta de productos con los campos Nombre, Precio y Cantidad, junto con el boton 'Agregar Producto' en color verde. Debajo aparece la barra de busqueda con los botones Buscar (azul), Registrar Venta (naranja) y Eliminar (rojo). En la parte inferior se observan las dos tablas vacias: INVENTARIO y REGISTRO DE VENTAS.

Pantalla 2 – Agregar Producto

El usuario llena los tres campos del formulario (por ejemplo: Nombre = 'The Last of Us', Precio = '899.99', Cantidad = '5') y presiona 'Agregar Producto'. Aparece un cuadro de dialogo verde con el mensaje 'Producto agregado correctamente'. Tras confirmar, el producto queda visible en la tabla de inventario con sus datos.

Pantalla 3 – Tabla de Inventario con Productos

Una vez agregados varios productos, la tabla INVENTARIO muestra una fila por cada producto con tres columnas: Nombre, Precio (con signo \$) y Cantidad. La tabla permite ver de un vistazo el stock disponible de toda la tienda.

Pantalla 4 – Registrar Venta

El usuario escribe el nombre del producto en el campo 'Producto:' y presiona 'Registrar Venta'. El sistema descuenta una unidad del inventario y registra la transaccion en la tabla REGISTRO DE VENTAS, mostrando el nombre del videojuego, su precio y la fecha y hora exacta de la venta (formato DD/MM/AAAA HH:MM:SS). Se muestra un mensaje de confirmacion 'Venta registrada correctamente'.

Pantalla 5 – Validacion y Manejo de Errores

Si el usuario intenta agregar un producto con campos vacios, el sistema muestra una advertencia en amarillo. Si el precio no es numerico, aparece un error en rojo. Si se intenta vender un producto con cantidad cero, se muestra 'Producto agotado'. Si se busca un nombre que no existe, aparece 'Producto no encontrado'. Estos controles garantizan la integridad de los datos en todo momento.

Nota: Las capturas del programa muestran siempre fondo blanco (#FFFFFF) tal como se configuro en ventana.config(bg='white'), cumpliendo con el lineamiento de fondo claro y buena legibilidad.

8. Conclusiones

¿Que aprendi?

El desarrollo de este proyecto me permitio consolidar los conocimientos adquiridos durante el curso de Introduccion a la Programacion. Aprendi a estructurar un programa completo desde cero: definir el problema, disenar la logica, implementar las funciones y construir una interfaz grafica funcional con Tkinter. Tambien profundice en el manejo de estructuras de datos como listas de diccionarios, que son fundamentales para organizar informacion compleja en Python.

Dificultades encontradas

Las principales dificultades fueron el diseño de la interfaz grafica con Tkinter, especialmente la disposicion de los elementos con los gestores de layout pack() y grid(). Tambien fue retador garantizar que las tablas (Treeview) se actualizaran correctamente despues de cada operacion, limpiando los registros anteriores antes de recargar los datos.

¿Como resolví los problemas?

Cada dificultad se abordó de manera metodica: primero comprendi el problema, luego busque la solucion en la documentacion oficial de Python y Tkinter, la probe en fragmentos pequenos y finalmente la integre al codigo principal. La validacion de tipos de datos se resolvió con bloques try/except, y la actualizacion de tablas se soluciono con la instruccion delete(*tabla.get_children()) antes de recargar.

Importancia de la programacion en mi carrera

Como estudiante de Ingenieria de Software, este proyecto demuestra que la programacion no es solo escribir codigo: es analizar problemas reales y construir soluciones eficientes. Un ingeniero de software debe ser capaz de traducir las necesidades de un cliente o negocio en software funcional. Este sistema de tienda es un ejemplo pequeno pero real de lo que se hace profesionalmente en el area.

Uso de nuevas tecnologias

Durante el desarrollo del proyecto se utilizo Inteligencia Artificial como herramienta de apoyo para resolver dudas puntuales de sintaxis y validar conceptos. Sin embargo, la logica del programa, su estructura y su implementacion fueron realizadas de forma propia, comprendiendo cada linea de codigo. El uso de IA como apoyo es una habilidad importante para el ingeniero moderno, siempre que se acompañe de comprension real del problema.

"Aqui el estudiante deja de resolver ejercicios y comienza a desarrollar soluciones reales como ingeniero."

— Estilo ChagoUaz —
